

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA6502	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	S. Muskan	Reg. No.:	192472182
Section / Batch:	BE-CSE-AI	Date:	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

*I, **S. Muskan (192472182)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **S. Muskan**

SECTION A – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

2. AI Text Summarizer: Develop an application that accepts a long paragraph and uses an LLM API to generate a concise summary. Demonstrate short, long, and empty inputs.

22. ChromaDB-Based Knowledge Search: Create a ChromaDB collection containing at least 10 documents and retrieve relevant documents based on a user's question.

QUESTION 2 – AI Text Summarizer

Aim

To develop a Python application that accepts a long paragraph and uses an LLM API to generate a concise summary.

Platform Used

Google Gemini API

Requirements

Python, Google Gemini API, and the google-genai Python package.

Description

The application accepts text from the user and sends the input to the Google Gemini large language model. The model generates a concise summary while preserving the important ideas and facts from the original text.

Implementation

The implementation uses a modular summarization function. The program first validates the input, checks whether the Gemini API key is available, sends a summarization prompt to Gemini, checks the returned response, and displays the generated summary. Exceptions are handled so that API failures and other runtime errors are reported clearly.

Program Flow

Accept paragraph from the user.

Validate the input and check for empty text.

Load and validate the Gemini API key.

Send the text and summarization prompt to the Gemini API.

Receive the generated summary.

Display the summary to the user.

Handle API and other runtime errors.

Code

```
from google import genai

def summarize_text(text):
    """Generate a concise summary using Google Gemini."""

    if not text or not text.strip():
        return "Error: Input text cannot be empty."

    if not GEMINI_API_KEY:
        return "Error: GEMINI_API_KEY is not configured."

    try:
        client = genai.Client(api_key=GEMINI_API_KEY)

        prompt = f"""
Summarize the following text concisely.
Keep the important ideas and facts.
Do not add information that is not present in the original text.

TEXT:
{text}
"""

        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=prompt
        )

        if not response.text:
            return "Error: Gemini returned an empty response."

        return response.text.strip()

    except Exception as e:
        return f"API Error: {type(e).__name__}: {e}"

def run_summarizer_test(test_name, text):
    print("=" * 70)
    print(test_name)
    print("=" * 70)
    print("\nINPUT:")
    print(text if text.strip() else "[EMPTY INPUT]")
    print("\nGENERATED SUMMARY:")
    print(summarize_text(text))
    print()
```

Test Case 1 – Short Input

Input:

Artificial Intelligence enables computers to perform tasks that normally require human intelligence.

Output:

```
=====
TEST CASE 1: SHORT INPUT
=====

INPUT:
Artificial Intelligence enables computers to perform tasks that normally require human intelligence.

GENERATED SUMMARY:
Artificial Intelligence allows computers to perform tasks that typically require human intelligence.
```

Test Case 2 – Long Input

Input:

Artificial Intelligence is a rapidly developing field of computer science that focuses on creating systems capable of performing tasks that normally require human intelligence. Machine learning is an important part of AI that allows computers to learn patterns from data without being explicitly programmed for every task. Deep learning uses neural networks with multiple layers to process complex information such as images, speech, and text. AI is now used in healthcare, finance, transportation, education, agriculture, and many other industries. In healthcare, AI can assist doctors with medical image analysis and prediction. In finance, it can help detect fraudulent transactions and analyze financial patterns. Although AI provides many benefits, it also introduces challenges such as privacy concerns, algorithmic bias, security risks, job displacement, and the need for responsible development. Therefore, AI systems should be developed and deployed with appropriate safety, transparency, fairness, and human oversight.

Output:

```
=====
TEST CASE 2: LONG INPUT
=====

INPUT:

Artificial Intelligence is a rapidly developing field of computer science
that focuses on creating systems capable of performing tasks that normally
require human intelligence. Machine learning is an important part of AI that
allows computers to learn patterns from data without being explicitly
programmed for every task. Deep learning uses neural networks with multiple
layers to process complex information such as images, speech, and text.
AI is now used in healthcare, finance, transportation, education, agriculture,
and many other industries. In healthcare, AI can assist doctors with medical
image analysis and prediction. In finance, it can help detect fraudulent
transactions and analyze financial patterns. Although AI provides many
benefits, it also introduces challenges such as privacy concerns, algorithmic
bias, security risks, job displacement, and the need for responsible
development. Therefore, AI systems should be developed and deployed with
appropriate safety, transparency, fairness, and human oversight.

GENERATED SUMMARY:
Artificial Intelligence (AI) is a rapidly developing computer science field focused on creating systems that perform tasks typically requiring human intelligence. Machine learning, a core AI component, enables computers
```

Test Case 3 – Empty Input

Input: Empty input

Output:

```
=====
TEST CASE 3: EMPTY INPUT
=====

INPUT:
[EMPTY INPUT]

GENERATED SUMMARY:
Error: Input text cannot be empty.
```

Error and Edge Case Handling

Empty or whitespace-only input is rejected.

Missing Gemini API key is reported clearly.

API exceptions are caught and displayed.

An empty API response is handled.

Valid input is processed normally.

Result

The AI Text Summarizer was successfully implemented using Python and the Google Gemini API. The application accepts user input, generates a concise summary, handles invalid input and API errors, and was demonstrated using short, long, and empty inputs.

QUESTION 22 – ChromaDB-Based Knowledge Search

Aim

To create a ChromaDB collection containing at least 10 documents and retrieve relevant documents based on a user's question.

Technology Used

Python and ChromaDB

Description

The application creates a ChromaDB collection containing 10 knowledge documents related to artificial intelligence and computer science. When the user enters a question, ChromaDB performs vector similarity search and returns the most relevant documents.

Knowledge Base

Document 1: Artificial Intelligence enables computers to perform tasks that normally require human intelligence.

Document 2: Machine Learning allows computers to learn patterns from data and make predictions without explicit programming.

Document 3: Deep Learning uses multi-layer neural networks to learn complex patterns from large datasets.

Document 4: Natural Language Processing enables computers to understand, process, and generate human language.

Document 5: Computer Vision allows computers to understand and analyze images and videos.

Document 6: Python is a popular programming language used extensively in artificial intelligence, machine learning, and data science.

Document 7: SQL is a language used to store, retrieve, and manipulate data in relational databases.

Document 8: Cloud computing provides computing resources such as servers, storage, and databases through the internet.

Document 9: Cybersecurity protects computer systems, networks, and data from unauthorized access and attacks.

Document 10: Data Science combines statistics, programming, and machine learning to extract useful insights from data.

Retrieval Process

Create the ChromaDB client and collection.

Add the 10 knowledge documents to the collection.

ChromaDB generates vector representations for the stored documents.

Accept a question from the user.

Convert the query into a vector representation.

Perform similarity search against the stored document vectors.

Retrieve the top 3 most relevant documents.

Display document IDs, distances, and retrieved document content.

Retrieval Pipeline: User Question → Query Embedding → Similarity Search → Top-K Relevant Documents → Display Retrieved Results

Code

```
def search_knowledge(question, n_results=3):
    """Retrieve the most relevant documents for a user question."""

    if not question or not question.strip():
        print("Error: Question cannot be empty.")
        return

    if n_results < 1:
        print("Error: n_results must be at least 1.")
        return

    try:
        # ChromaDB converts the query into an embedding and
        # retrieves the nearest documents from the collection.
        results = collection.query(
            query_texts=[question],
            n_results=min(n_results, collection.count())
        )

        retrieved_ids = results["ids"][0]
        retrieved_docs = results["documents"][0]
        distances = results["distances"][0]

        print("=" * 70)
        print("RETRIEVAL RESULTS")
        print("=" * 70)
        print("Question:", question)
        print("Number of retrieved documents:", len(retrieved_docs))

        for i, (doc_id, doc, distance) in enumerate(
            zip(retrieved_ids, retrieved_docs, distances), 1
        ):
            print(f"\nResult {i}")
            print(f"Document ID : {doc_id}")
            print(f"Distance      : {distance:.4f}")
            print(f"Document       : {doc}")

        print("=" * 70)

    except Exception as e:
        print(f"Search Error: {type(e).__name__}: {e}")
```

Test Case 1 – Machine Learning

Input:

What is machine learning and how does it learn from data?

Output:

```
=====
RETRIEVAL RESULTS
=====
Question: What is machine learning and how does it learn from data?
Number of retrieved documents: 3

Result 1
Document ID : doc_2
Distance    : 0.6140
Document    : Machine Learning allows computers to learn patterns from data and make predictions without explicit programming.

Result 2
Document ID : doc_10
Distance    : 0.7411
Document    : Data Science combines statistics, programming, and machine learning to extract useful insights from data.

Result 3
Document ID : doc_1
Distance    : 1.1528
Document    : Artificial Intelligence enables computers to perform tasks that normally require human intelligence.
=====
```

Test Case 2 – Computer Vision

Input:

How can computers analyze images and videos?

Output:

```
=====
RETRIEVAL RESULTS
=====
Question: How can computers analyze images and videos?
Number of retrieved documents: 3

Result 1
Document ID : doc_5
Distance    : 0.6133
Document    : Computer Vision allows computers to understand and analyze images and videos.

Result 2
Document ID : doc_4
Distance    : 1.2866
Document    : Natural Language Processing enables computers to understand, process, and generate human language.

Result 3
Document ID : doc_10
Distance    : 1.3194
Document    : Data Science combines statistics, programming, and machine learning to extract useful insights from data.
=====
```

Test Case 3 – Cybersecurity

Input:

What protects computer networks from unauthorized access?

Output:

```
=====
RETRIEVAL RESULTS
=====
Question: What protects computer networks from unauthorized access?
Number of retrieved documents: 3

Result 1
Document ID : doc_9
Distance    : 0.6097
Document    : Cybersecurity protects computer systems, networks, and data from unauthorized access and attacks.

Result 2
Document ID : doc_5
Distance    : 1.5278
Document    : Computer Vision allows computers to understand and analyze images and videos.

Result 3
Document ID : doc_8
Distance    : 1.5914
Document    : Cloud computing provides computing resources such as servers, storage, and databases through the internet.
=====
```

Test Case 4 – Empty Question

Input:

Empty input

Output:

```
question_4 = ""

search_knowledge(question_4, n_results=3)

Error: Question cannot be empty.
```

Error and Edge Case Handling

Empty or whitespace-only questions are rejected.

The number of requested results is validated.

ChromaDB search exceptions are caught and displayed.

The program limits the number of results to the number of documents available.

Valid questions return the most relevant documents.

Result

The ChromaDB-based knowledge search system was successfully implemented using Python. A collection containing 10 documents was created, and relevant documents were retrieved using vector similarity search based on the user's question. The retrieval process was displayed clearly, and the system was tested with multiple questions and an empty input.

Overall Conclusion

Both assigned AI applications were implemented using Python. The first application demonstrates LLM-based text summarization using Google Gemini, while the second demonstrates vector-database-based knowledge retrieval using ChromaDB. Both programs include user input handling, error and edge-case handling, modular implementation, and suitable test cases.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
API Integration Correctness	30%	Correct, efficient API integration; understands request/response flow	Correct integration with minor inefficiencies	Partial integration with functional errors	API integration non-functional / major errors
Problem-Solving Completeness	25%	Solves the given problem completely and robustly	Solves problem with minor issues	Partial solution to the problem	Does not solve the assigned problem
Code Quality	20%	Clean, well-structured, error-handled code	Reasonably clean code	Code works but poorly structured	Code is unreliable or unstructured
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases	Demo covers the main use cases	Demo is incomplete	No functional demo presented
Code Documentation	10%	Well-commented code with clear README/setup steps	Adequate comments/instructions	Minimal documentation provided	No documentation provided

Marks Evaluation:

Question No.	API Integration Correctness(30%)	Problem-Solving Completeness (25%)	Code Quality(20%)	Input/Output Demo(15%)	Code Documentation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						